PRACTICALWEEK11

TASK1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
  GNU nano 7.2                                                  race.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        int temp = counter;
        usleep(1);    // Delay to make race condition visible
        counter = temp + 1;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Initial counter value: %d\n", counter);

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
           NUM_THREADS * INCREMENTS);

    printf("Actual counter value: %d\n", counter);

    return 0;
}
```

```
manikanta@Hemu: ~/Practical11
manikanta@Hemu:~/Practical11$ mkdir Practicall11
manikanta@Hemu:~/Practical11$ cd Practical11
manikanta@Hemu:~/Practical11/Practical11$ nano race.c
manikanta@Hemu:~/Practical11/Practical11$ gcc race.c -o race -pthread
manikanta@Hemu:~/Practical11/Practical11$ ./race
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 101080
manikanta@Hemu:~/Practical11/Practical11$
```

TASK2: Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
  GNU nano 7.2                                  mutex.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;

pthread_mutex_t lock;

void *increment_counter(void *arg)
{
     for (int i = 0; i < INCREMENTS; i++)
     {
          pthread_mutex_lock(&lock);

          int temp = counter;
          usleep(1);
          counter = temp + 1;

          pthread_mutex_unlock(&lock);
     }

     return NULL;
}

int main()
{
     pthread_t threads[NUM_THREADS];
```

```
  GNU nano 7.2                          mutex.c *
{
    pthread_t threads[NUM_THREADS];

    // Initialize mutex
    pthread_mutex_init(&lock, NULL);

    printf("Initial counter value: %d\n", counter);

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

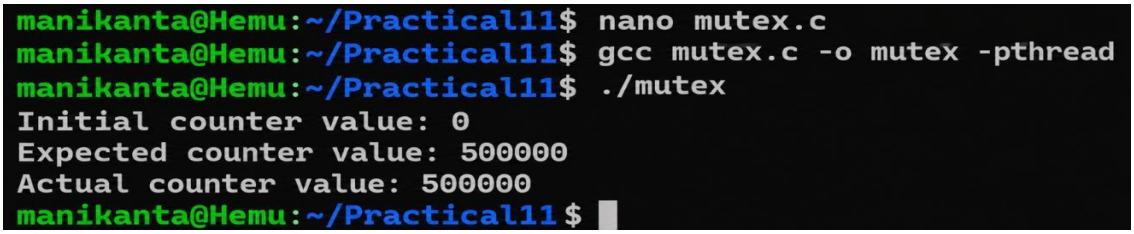
    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
           NUM_THREADS * INCREMENTS);

    printf("Actual counter value: %d\n", counter);

    // Destroy mutex
    pthread_mutex_destroy(&lock);

    return 0;
}
```


manikanta@Hemu:~/Practical11$ nano mutex.c
manikanta@Hemu:~/Practical11$ gcc mutex.c -o mutex -pthread
manikanta@Hemu:~/Practical11$ ./mutex
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 500000
manikanta@Hemu:~/Practical11$